

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

TR-AURA-001: AURA Input Payload Processing

Related Functional Requirement(s): FR-AURA-001

Objective: Implement a data processing pipeline to decode, normalize, and transform AURA input payloads into a canonical XML string.

Target Cluster Configuration:

- Cluster Name: Data Processing Cluster
- Databricks Runtime Version: 7.3 LTS
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 (autoscaling)
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: XML parsing library, JSON parsing library, XSLT transformation library

Source Data Details:

- I_AURA_INPUT_BASE64: Input data stored in a message queue
- POLICY_NUMBER: Input data stored in a database table

Target Data Details:

- PARSED_AURA_OUTPUT: Canonical XML string stored in a database table or file system

Job Flow / Pipeline Stages:

- Read I_AURA_INPUT_BASE64 and POLICY_NUMBER inputs
- Base64-decode I_AURA_INPUT_BASE64
- Parse JSON and extract interviewDetails (if necessary)
- Perform progressive decoding loop until XML is obtained
- Apply XSLT transformation based on Aura version
- Remove XML declaration encoding attribute and wrap content in "XMLA" container element
- Instantiate Transformer and transform wrapped XML string to produce canonical output

Data Transformations / Business Logic:

- Step 1: Base64-decode I_AURA_INPUT_BASE64

- Transformation Logic: Use a Base64 decoding library to decode the input string
- Step 2: Parse JSON and extract interviewDetails (if necessary)
- Transformation Logic: Use a JSON parsing library to parse the input string and extract the required data
- Step 3: Perform progressive decoding loop until XML is obtained
- Transformation Logic: Use a loop to iteratively decode the input string until it starts with "<"
- Step 4: Apply XSLT transformation based on Aura version
- Transformation Logic: Use an XSLT transformation library to apply the required transformation based on the Aura version

Error Handling and Logging:

- Log errors and exceptions during AURA parsing, XSLT transformation, and output generation
- Increment error count and allow session to continue unless threshold is exceeded
- Provide reject target or error file for capturing error details

TR-FILE-001: 10K Flat File Generation

Related Functional Requirement(s): FR-FILE-001

Objective: Implement a data processing pipeline to generate a 10K flat file with derived fields and filename.

Target Cluster Configuration:

- Cluster Name: Data Processing Cluster
- Databricks Runtime Version: 7.3 LTS
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 (autoscaling)
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: Data processing library, filename generation library

Source Data Details:

- T_APPLICATION_STAGING: Source data stored in a database table
- T_BENEFICIARY: Source data stored in a database table

Target Data Details:

- File_Structure_10K: 10K flat file stored in a file system

Job Flow / Pipeline Stages:

- Retrieve data from T_APPLICATION_STAGING and T_BENEFICIARY using \$\$M_SRC_SQL
- Derive Field_1..Field_n using mplt_BNCPLS_IF23_10K_Field_Calc mapplet

- Derive File_Path_Name using EXP_Calc_10K_FileName expression
- Apply Transaction Control (TC_Seperate_10K_Files) to split output into multiple files
- Write derived fields and filename to File_Structure_10K flat file target

Data Transformations / Business Logic:

- Step 1: Derive Field_1..Field_n using mplt_BNCPLS_IF23_10K_Field_Calc mapplet
- Transformation Logic: Use a data processing library to derive the required fields using the mapplet
- Step 2: Derive File_Path_Name using EXP_Calc_10K_FileName expression
- Transformation Logic: Use a filename generation library to derive the required filename using the expression

Error Handling and Logging:

- Log errors and exceptions during data processing, filename generation, and file writing
- Increment error count and allow session to continue unless threshold is exceeded
- Provide reject target or error file for capturing error details

TR-TRANS-001: Transaction Control

Related Functional Requirement(s): FR-TRANS-001

Objective: Implement transaction control to split output into multiple files.

Target Cluster Configuration:

- Cluster Name: Data Processing Cluster
- Databricks Runtime Version: 7.3 LTS
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 (autoscaling)
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: Transaction control library

Job Flow / Pipeline Stages:

- Group rows into transactions based on \$\$MAX_ROWS_PER_FILE
- Commit and initiate new output file context when threshold is reached
- Ensure each file has a unique filename

Data Transformations / Business Logic:

- Step 1: Group rows into transactions based on \$\$MAX_ROWS_PER_FILE
- Transformation Logic: Use a transaction control library to group rows into transactions based on the threshold

Error Handling and Logging:

- Log errors and exceptions during transaction control and file writing
- Increment error count and allow session to continue unless threshold is exceeded
- Provide reject target or error file for capturing error details

TR-ERR-001: Error Handling and Logging**Related Functional Requirement(s): FR-ERR-001**

Objective: Implement error handling and logging for AURA parsing, mapplet execution, and file writing.

Error Handling and Logging:

- Log errors and exceptions during AURA parsing, mapplet execution, and file writing
- Increment error count and allow session to continue unless threshold is exceeded
- Provide reject target or error file for capturing error details